



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

ZIPKIN PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Observability

TOTAL: 10 QUESTIONS

Q1 What is Zipkin and what problem in Observability does it solve? [What Model](#)

Q6 How does Zipkin handle reliability and high availability? [Observability](#)

Q2 What are the core components or concepts in Zipkin? [Architecture](#)

Q7 What observability does Zipkin provide out of the box? [Lifecycle](#)

Q3 How does Zipkin compare to its main alternatives? [Configuration](#)

Q8 What are common performance bottlenecks in Zipkin? [Compliance](#)

Q4 How do you get started with Zipkin for a new project? [Hardening](#)

Q9 What security best practices apply when running Zipkin? [Testing](#)

Q5 What configuration options most affect Zipkin's behavior in production? [Tracing / IDP](#)

Q10 What mistakes do teams commonly make when adopting Zipkin? [Tuning](#)



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Zipkin is a tool or platform that

Mitigation

Zipkin is a tool or platform that automates or simplifies a specific concern in the Observability domain, reducing manual effort and operational complexity for engineering teams.

A2 Zipkin has key building blocks that work

Primitives

Zipkin has key building blocks that work together: a configuration layer defines intent, a runtime layer enforces it, and an API or CLI layer allows operators to manage and observe the system.

A3 Zipkin trades off simplicity against feature richness

Hardening

Zipkin trades off simplicity against feature richness differently than its alternatives. Choose Zipkin when its specific strengths performance, ecosystem, or ease of use align with your team's primary constraints.

A4 Install Zipkin via its package manager or

Gotchas

Install Zipkin via its package manager or binary release. Follow the quickstart to initialize a project, configure the minimal required settings, and run the default command to verify the setup works end-to-end.

A5 Production-critical settings typically include concurrency

Concurrency

Production-critical settings typically include concurrency limits, timeout values, retry policies, resource limits, and logging levels. Review the official production hardening guide before deploying.

A6 Zipkin provides HA through leader election, replication, or stateless horizontal

Observability

scaling. Deploy at least two instances across availability zones and configure health checks for automatic failover.

A7 Zipkin exposes metrics (Prometheus endpoint or built-in dashboard), structured

Information

logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

A8 Performance bottlenecks typically stem from misconfigured connection

Performance

Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

A9 Run Zipkin with the principle of least

Assessment

Run Zipkin with the principle of least privilege, enable TLS for all communication, use a secrets manager for credentials, and keep the software patched to the latest stable release.

A10 Common pitfalls include skipping the documentation and

Configuration

Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.